Supplementary Analysis: Account-Level Survival Structure

Sparkov Credit Card Transactions Dataset

Arjun Sekhar

2026-10-04

Contents

1 Introduction **2**
1.1 Objective . . . 2
1.2 Motivation . . . 2

2 Libraries and Data **2**
2.1 Load Packages and Set Seed . . . 2
2.2 Load and Structure-Check the Data . . . 2

3 Account-Level Survival Dataset Construction **4**
3.1 Censoring Window Selection . . . 4
3.2 Diagnostic: The Injected-Account Artefact . . . 6
3.3 Final Dataset Construction and Sanity Checks . . . 9
3.4 Diagnostic: Fraud Structure Checks . . . 11

4 Model Comparison: RSF vs Cox vs LR vs RF **13**
4.1 Full-Data Fits . . . 13
4.2 Random Forest (Classification Control) . . . 16
4.3 Repeated Stratified Evaluation (30 Splits) . . . 17
4.4 Concordance Direction Bug: Diagnostic . . . 18
4.5 Results Summary . . . 19
4.6 Cross-Model Explainability . . . 20
4.7 Fraud Amount Structure . . . 23

5 Aggregate Poisson/NB Analysis (Harmonic) **24**
5.1 Design: Defining "Aggregate" for Account-Level Data . . . 24
5.2 Model Fitting . . . 25
5.3 Diagnostic: Fraud Burst Concentration and Volume Structure . . . 28
5.4 Comparison to ULB Diurnal Findings . . . 33

6 Synthesis **33**
6.1 Summary of Model Performance . . . 33
6.2 Connection to the Conditions Matrix and DGP Simulation . . . 33
6.3 Status and Open Items . . . 34

1 Introduction

1.1 Objective

This notebook documents a supplementary analysis testing whether the Random Survival Forest (RSF) shows a genuine performance advantage over parametric and classification alternatives when evaluated on data with genuine account-level survival structure, rather than the transaction-level structure of the ULB dataset used throughout the main thesis. This was raised directly by Maurizio following review of the diurnal time-structure analysis, and reflects a concern present since early supervisor meetings: that the null result on ULB may reflect the absence of the structural conditions RSF is designed to exploit, rather than a genuine absence of RSF's advantage as a method.

1.2 Motivation

The main thesis establishes, through the conditions matrix (Baralou et al., 2023; Omurlu et al., 2009; Graf et al., 2025) and a controlled data-generating process simulation, that RSF's null result on ULB is predictable from that dataset's structural characteristics - specifically, its PCA-orthogonalised predictor structure, extreme censoring (99.83%), and modest event count. The DGP Scenario B simulation confirmed that RSF recovers its expected advantage under moderate censoring (~50%) and genuine covariate interactions.

This notebook provides the empirical, real-data counterpart to that simulation. The Sparkov-generated credit card transactions dataset (Kaggle: `kartik2112/fraud-detection`) offers a persistent account identifier (`cc_num`) with repeated transactions per account over a two-year window, allowing construction of a genuine time-to-first-fraud-event variable per account, with right-censoring for accounts that do not experience fraud within the observation window - the survival structure ULB's transaction-level format cannot support.

2 Libraries and Data

2.1 Load Packages and Set Seed

```
## Step 1: Load required packages
library(dplyr)
library(lubridate)
library(pander)

## Step 2: Set seed
set.seed(44940076)
```

2.2 Load and Structure-Check the Data

```
## Step 1: Load train/test files
train <- read.csv("fraudTrain.csv", header = TRUE)
test  <- read.csv("fraudTest.csv", header = TRUE)

## Step 2: Structure check - confirm actual column names before assuming anything
dim(train)
```

```
## [1] 1296675      23
```

```
str(train)
```

```
## 'data.frame':    1296675 obs. of  23 variables:
##  $ X                    : int  0 1 2 3 4 5 6 7 8 9 ...
##  $ trans_date_trans_time: chr  "2019-01-01 00:00:18" "2019-01-01 00:00:44" "2019-01-01 00:00:51" "201
##  $ cc_num               : num  2.70e+15 6.30e+11 3.89e+13 3.53e+15 3.76e+14 ...
```

```
##  $ merchant            : chr  "fraud_Rippin, Kub and Mann" "fraud_Heller, Gutmann and Zieme" "fraud
##  $ category            : chr  "misc_net" "grocery_pos" "entertainment" "gas_transport" ...
##  $ amt                 : num  4.97 107.23 220.11 45 41.96 ...
##  $ first               : chr  "Jennifer" "Stephanie" "Edward" "Jeremy" ...
##  $ last                : chr  "Banks" "Gill" "Sanchez" "White" ...
##  $ gender              : chr  "F" "F" "M" "M" ...
##  $ street              : chr  "561 Perry Cove" "43039 Riley Greens Suite 393" "594 White Dale Suite
##  $ city                : chr  "Moravian Falls" "Orient" "Malad City" "Boulder" ...
##  $ state               : chr  "NC" "WA" "ID" "MT" ...
##  $ zip                 : int  28654 99160 83252 59632 24433 18917 67851 22824 15665 37040 ...
##  $ lat                 : num  36.1 48.9 42.2 46.2 38.4 ...
##  $ long                : num  -81.2 -118.2 -112.3 -112.1 -79.5 ...
##  $ city_pop            : int  3495 149 4154 1939 99 2158 2691 6018 1472 151785 ...
##  $ job                 : chr  "Psychologist, counselling" "Special educational needs teacher" "Natu
##  $ dob                 : chr  "1988-03-09" "1978-06-21" "1962-01-19" "1967-01-12" ...
##  $ trans_num           : chr  "0b242abb623afc578575680df30655b9" "1f76529f8574734946361c461b024d99"
##  $ unix_time           : int  1325376018 1325376044 1325376051 1325376076 1325376186 1325376248 132
##  $ merch_lat           : num  36 49.2 43.2 47 38.7 ...
##  $ merch_long          : num  -82 -118.2 -112.2 -112.6 -78.6 ...
##  $ is_fraud            : int  0 0 0 0 0 0 0 0 0 0 ...
```

```
## Step 3: Confirm account identifier behaviour - checking cc_num gives clean,
## stable per-account grouping across the two files
n_distinct(train$cc_num)
```

```
## [1] 983
```

```
n_distinct(test$cc_num)
```

```
## [1] 924
```

```
length(intersect(unique(train$cc_num), unique(test$cc_num)))
```

```
## [1] 908
```

```
## Step 4: Date range and fraud rate - confirms whether recombining train/test
## preserves a clean chronological per-account timeline
range(train$trans_date_trans_time)
```

```
## [1] "2019-01-01 00:00:18" "2020-06-21 12:13:37"
```

```
range(test$trans_date_trans_time)
```

```
## [1] "2020-06-21 12:14:25" "2020-12-31 23:59:34"
```

```
mean(train$is_fraud)
```

```
## [1] 0.005788652
```

```
mean(test$is_fraud)
```

```
## [1] 0.003859864
```

```
## Step 5: Per-account transaction counts and fraud incidence at full length -
## this tells us how many accounts have a fraud event vs how many are fully
## censored across the entire two-year window, before any windowing decision
account_summary <- bind_rows(train, test) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(
    n_transactions = n(),
```

```
    n_fraud        = sum(is_fraud),
    first_txn      = min(trans_date_trans_time),
    last_txn       = max(trans_date_trans_time),
    .groups = "drop"
  )

nrow(account_summary)
```

```
## [1] 999
```

```
mean(account_summary$n_fraud > 0)
```

```
## [1] 0.976977
```

```
summary(account_summary$n_transactions)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       6     740    1471    1854    2917    4392
```

Observations: The `cc_num` identifier behaves cleanly as a stable account grouping variable, with 908 of 999 accounts persisting across both the train and test files, and the test file's date range beginning within seconds of where the train file ends (`2020-06-21 12:13:37` to `12:14:25`), confirming that recombining the two files produces a genuinely continuous per-account timeline. At full two-year length, however, 97.7% of accounts experience at least one fraud event, leaving only 2.3% censored - the near-opposite extreme from ULB's 99.83% censoring, and a consequence of the generator assigning a fraud episode to approximately 98% of customers (Section 3.4), rather than a meaningful account-risk signal. This motivates the windowing decision in the following section, since a dataset with almost no censoring gives RSF little of the structure it is intended to exploit either.

3 Account-Level Survival Dataset Construction

3.1 Censoring Window Selection

```
## Step 1: Recombine and parse dates, explicitly forcing UTC to sidestep any
## local-timezone/DST parsing issues
full_data <- bind_rows(train, test) %>%
  dplyr::mutate(trans_date_trans_time = as.POSIXct(trans_date_trans_time, format = "%Y-%m-%d %H:%M:%S",

## Step 2: Check for parse failures directly - rules out a parsing bug before
## trusting anything downstream
sum(is.na(full_data$trans_date_trans_time))
```

```
## [1] 0
```

```
n_distinct(full_data$cc_num)
```

```
## [1] 999
```

```
## Step 3: Compute each account's FULL observation span (unfiltered) - this is
## what determines true right-censoring, not a windowed subset
account_full_range <- full_data %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(
    t0 = min(trans_date_trans_time, na.rm = TRUE),
    t_last_observed = max(trans_date_trans_time, na.rm = TRUE),
    total_days_observed = as.numeric(difftime(t_last_observed, t0, units = "days")),
```

```
    .groups = "drop"
  )

nrow(account_full_range)
```

```
## [1] 999
```

```
## Step 4: Censoring rate under candidate windows - censoring status determined
## by whether fraud occurred within the window AND whether the account's real
## history extends past the window (vs the account's data simply running out)
full_data <- full_data %>%
  dplyr::left_join(account_full_range %>% dplyr::select(cc_num, t0), by = "cc_num") %>%
  dplyr::mutate(days_since_start = as.numeric(difftime(trans_date_trans_time, t0, units = "days")))

check_window_v2 <- function(window_days) {
  full_data %>%
    dplyr::filter(days_since_start <= window_days) %>%
    dplyr::group_by(cc_num) %>%
    dplyr::summarise(had_fraud_in_window = any(is_fraud == 1, na.rm = TRUE), .groups = "drop") %>%
    dplyr::left_join(account_full_range, by = "cc_num") %>%
    dplyr::mutate(account_data_covers_full_window = total_days_observed >= window_days) %>%
    dplyr::summarise(
      n_accounts    = n(),
      pct_event     = mean(had_fraud_in_window) * 100,
      pct_censored  = 100 - pct_event,
      pct_data_covers_window = mean(account_data_covers_full_window, na.rm = TRUE) * 100
    )
}

window_3mo <- check_window_v2(90)
window_6mo <- check_window_v2(182)

pander(window_3mo, caption = "90-Day Observation Window - Censoring Summary")
```

Table 1: 90-Day Observation Window - Censoring Summary

n_accounts	pct_event	pct_censored	pct_data_covers_window
999	21.92	78.08	90.89

```
pander(window_6mo, caption = "182-Day Observation Window - Censoring Summary")
```

Table 2: 182-Day Observation Window - Censoring Summary

n_accounts	pct_event	pct_censored	pct_data_covers_window
999	32.83	67.17	90.89

Observations: The 90-day window yields 78.1% censoring (21.9% event rate), while the 182-day window yields 67.2% censoring (32.8% event rate), with identical data coverage (90.9%) across both - meaning the extension to 182 days costs nothing in terms of accounts lacking sufficient history, while providing substantially more events for a stable comparison. The 182-day censoring rate of 67.2% sits considerably closer to the moderate-censoring condition (~50%) under which the DGP Scenario B simulation already demonstrated an RSF advantage than to ULB's extreme 99.83% censoring, providing a literature-grounded justification

for this window choice rather than an arbitrary one. The 182-day window was therefore selected for the account-level survival dataset constructed below.

3.2 Diagnostic: The Injected-Account Artefact

```
## Step 1: Build the account-level covariate set from transactions within
## each account's 182-day observation window
window_days <- 182

survival_data <- full_data %>%
  dplyr::filter(days_since_start <= window_days) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(
    event         = as.integer(any(is_fraud == 1, na.rm = TRUE)),
    time_to_event = dplyr::if_else(
      any(is_fraud == 1, na.rm = TRUE),
      min(days_since_start[is_fraud == 1], na.rm = TRUE),
      max(days_since_start, na.rm = TRUE)
    ),
    gender         = dplyr::first(gender),
    dob            = dplyr::first(dob),
    city_pop       = dplyr::first(city_pop),
    state          = dplyr::first(state),
    n_transactions = n(),
    mean_amt       = mean(amt, na.rm = TRUE),
    sd_amt         = sd(amt, na.rm = TRUE),
    max_amt        = max(amt, na.rm = TRUE),
    n_categories   = n_distinct(category),
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    age    = as.numeric(difftime(as.Date("2019-01-01"), as.Date(dob), units = "days")) / 365.25,
    sd_amt = dplyr::if_else(is.na(sd_amt), 0, sd_amt),
    gender = factor(gender)
  )

## Step 2: Sanity checks before trusting this dataset
dim(survival_data)
```

```
## [1] 999  13
```

```
sum(survival_data$event)
```

```
## [1] 328
```

```
summary(survival_data$time_to_event)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##     0.0   117.1   181.4   142.3   181.8   182.0
```

```
sum(is.na(survival_data))
```

```
## [1] 0
```

```
pander(head(survival_data, 10), caption = "Account-Level Survival Dataset (Pre-Diagnostic) - First 10 R
```

Table 3: Account-Level Survival Dataset (Pre-Diagnostic) - First 10 Rows (continued below)

cc_num	event	time_to_event	gender	dob	city_pop	state
6.042e+10	1	58.53	F	1986-02-17	1645	WY
6.042e+10	1	103.2	M	1942-04-02	46944	SC
6.042e+10	1	71.81	M	1958-07-28	83	OK
6.043e+10	1	104.8	M	1976-06-30	2142	OK
6.049e+10	0	181.2	M	1972-08-15	233060	MS
6.049e+10	0	181.6	F	1963-05-19	222	AR
6.05e+10	0	181	M	1942-11-24	1263321	TX
5.018e+11	0	181.1	M	1958-09-20	308	CA
5.018e+11	1	0	M	1941-07-06	172247	FL
5.018e+11	1	159.7	F	1958-10-26	602	KS

n_transactions	mean_amt	sd_amt	max_amt	n_categories	age
497	62.65	158.6	3075	14	32.87
499	76.05	137.7	1290	14	76.75
159	79.1	168.9	1222	14	60.43
173	141.8	210.6	1164	14	42.51
151	53.35	62.32	476	14	46.38
320	67.1	106.8	1145	14	55.62
141	61.6	75.56	509.5	14	76.1
474	60.69	127.4	1957	14	60.28
8	618.3	420.6	1084	3	77.49
179	75.69	142.8	984.8	13	60.18

```
## Step 1: 91 accounts show time_to_event == 0 - i.e. their first observed
## transaction in the window was already fraudulent. At an overall per-
## transaction fraud rate of ~0.5%, this is far too common to be chance,
## and is investigated directly rather than treated as a modelling nuisance.
sum(survival_data$time_to_event == 0)
```

```
## [1] 91
```

```
zero_time_accounts <- survival_data %>% dplyr::filter(time_to_event == 0) %>% pull(cc_num)

## Step 2: Do these accounts' t0 values cluster near the dataset's overall
## start date (which would suggest left-truncation - i.e. pre-existing
## accounts whose true history began before this dataset's recording window)?
full_data %>%
  dplyr::filter(cc_num %in% zero_time_accounts) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(t0 = min(trans_date_trans_time), .groups = "drop") %>%
  pull(t0) %>%
  summary()
```

```
##                  Min.                1st Qu.                  Median
## "2019-01-02 01:47:29" "2019-04-08 22:45:10" "2019-12-04 00:14:28"
##                  Mean                3rd Qu.                    Max.
## "2019-11-21 01:52:06" "2020-05-06 02:09:03" "2020-12-21 03:29:08"
```

```
## Step 3: Contrast against a random sample of non-zero-time accounts
full_data %>%
  dplyr::filter(!cc_num %in% zero_time_accounts) %>%
  dplyr::distinct(cc_num, .keep_all = FALSE) %>%
  dplyr::slice_sample(n = 91) %>%
  dplyr::left_join(full_data %>% dplyr::group_by(cc_num) %>% dplyr::summarise(t0 = min(trans_date_trans
  pull(t0) %>%
  summary()
```

```
##                  Min.               1st Qu.                Median
## "2019-01-01 00:00:18" "2019-01-01 02:23:11" "2019-01-01 06:35:07"
##                  Mean               3rd Qu.                  Max.
## "2019-01-01 13:46:14" "2019-01-01 15:19:02" "2019-01-05 06:15:05"
```

Observations: The left-truncation hypothesis is not supported by this contrast - if anything, the pattern runs the opposite direction. The 91 zero-time accounts' `t0` values are spread across the entire two-year window (median December 2019), while the comparison sample clusters almost entirely within the first days of January 2019. This rules out "pre-existing account whose history predates the dataset" as the explanation, and instead points toward these being accounts that enter the dataset partway through the window, with their very first recorded transaction already fraudulent - investigated further below.

```
## Step 1: Check whether "late-arriving account" (t0 well after dataset start)
## predicts zero-time-to-event, across ALL 999 accounts rather than a sample
account_full_range %>%
  dplyr::mutate(days_from_dataset_start = as.numeric(difftime(t0, as.POSIXct("2019-01-01", tz = "UTC"),
  pull(days_from_dataset_start) %>%
  summary()
```

```
##      Min.   1st Qu.    Median      Mean   3rd Qu.      Max.
##   0.00021   0.16288   0.42745  30.12915   0.86005 720.14523
```

```
## Step 2: Cross-tabulate directly - late arrivals defined as t0 more than
## 7 days after dataset start
late_arrival_summary <- account_full_range %>%
  dplyr::mutate(
    days_from_dataset_start = as.numeric(difftime(t0, as.POSIXct("2019-01-01", tz = "UTC"), units = "da
    late_arrival = days_from_dataset_start > 7
  ) %>%
  dplyr::left_join(survival_data %>% dplyr::select(cc_num, time_to_event), by = "cc_num") %>%
  dplyr::group_by(late_arrival) %>%
  dplyr::summarise(
    n_accounts    = n(),
    pct_zero_time = mean(time_to_event == 0, na.rm = TRUE) * 100,
    .groups = "drop"
  )

pander(late_arrival_summary, caption = "Zero-Time-to-Event Rate: Founding Cohort vs Late-Arriving Accou
```

Table 5: Zero-Time-to-Event Rate: Founding Cohort vs Late-Arriving Accounts

late_arrival	n_accounts	pct_zero_time
FALSE	906	0.1104
TRUE	93	96.77

Observations: The founding cohort (t0 within the first week) shows a zero-time-to-event rate of 0.11%, consistent with pure chance given the overall transaction-level fraud rate, while late-arriving accounts (t0 more than 7 days after dataset start) show a zero-time-to-event rate of 96.8%. This is close to deterministic, not merely elevated, and confirms these 93 accounts are not ordinary customer relationships that happen to start partway through the window, but represent a distinct structural pattern in how the dataset was generated.

```
## Step 1: Confirm late-arriving accounts are also short-lived overall,
## supporting an "injected event" rather than "ongoing relationship"
## interpretation
volume_summary <- account_full_range %>%
  dplyr::mutate(
    days_from_dataset_start = as.numeric(difftime(t0, as.POSIXct("2019-01-01", tz = "UTC"), units = "da
    late_arrival = days_from_dataset_start > 7
  ) %>%
  dplyr::left_join(survival_data %>% dplyr::select(cc_num, n_transactions), by = "cc_num") %>%
  dplyr::group_by(late_arrival) %>%
  dplyr::summarise(
    median_n_transactions = median(n_transactions, na.rm = TRUE),
    mean_n_transactions   = mean(n_transactions, na.rm = TRUE),
    .groups = "drop"
  )

pander(volume_summary, caption = "Transaction Volume: Founding Cohort vs Late-Arriving Accounts")
```

Table 6: Transaction Volume: Founding Cohort vs Late-Arriving Accounts

late_arrival	median_n_transactions	mean_n_transactions
FALSE	457.5	445.8
TRUE	10	14.92

Observations: Late-arriving accounts show a median of 10 transactions within the window, against 458 for the founding cohort, and their first recorded transaction almost always coincides with the start of their fraud episode. Their time origin is therefore defined by the fraud itself, and their sparse histories offer little for a model to learn from; including them would risk the models appearing to discriminate well by detecting this entry pattern rather than genuine account risk. These 93 accounts (9.3% of the original 999) are excluded from the survival dataset below. Section 3.4 examines them further and shows they are not fraud-only, so the exclusion rests on these data properties rather than on a known generator mechanism.

3.3 Final Dataset Construction and Sanity Checks

```
## Step 1: Rebuild the survival dataset restricted to the founding cohort,
## and replace n_categories (which saturates at Sparkov's 14-category maximum
## for 87.5% of accounts, and carries little discriminating power as a raw
## count) with a proportion-based entropy measure instead
founding_cohort <- account_full_range %>%
  dplyr::mutate(days_from_dataset_start = as.numeric(difftime(t0, as.POSIXct("2019-01-01", tz = "UTC"),
  dplyr::filter(days_from_dataset_start <= 7) %>%
  pull(cc_num)

survival_data_clean <- full_data %>%
  dplyr::filter(cc_num %in% founding_cohort, days_since_start <= window_days) %>%
```

```r
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(
    event         = as.integer(any(is_fraud == 1, na.rm = TRUE)),
    time_to_event = dplyr::if_else(
      any(is_fraud == 1, na.rm = TRUE),
      min(days_since_start[is_fraud == 1], na.rm = TRUE),
      max(days_since_start, na.rm = TRUE)
    ),
    gender         = dplyr::first(gender),
    dob            = dplyr::first(dob),
    city_pop       = dplyr::first(city_pop),
    state          = dplyr::first(state),
    n_transactions = n(),
    mean_amt       = mean(amt, na.rm = TRUE),
    sd_amt         = sd(amt, na.rm = TRUE),
    max_amt        = max(amt, na.rm = TRUE),
    category_entropy = {
      p <- table(category) / n()
      -sum(p * log(p))
    },
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    age    = as.numeric(difftime(as.Date("2019-01-01"), as.Date(dob), units = "days")) / 365.25,
    sd_amt = dplyr::if_else(is.na(sd_amt), 0, sd_amt),
    gender = factor(gender)
  )

## Step 2: Sanity checks on the final dataset
dim(survival_data_clean)
```

```
## [1] 906  13
```

```r
sum(survival_data_clean$event)
```

```
## [1] 237
```

```r
sum(survival_data_clean$time_to_event == 0)
```

```
## [1] 1
```

```r
pander(head(survival_data_clean, 10), caption = "Final Account-Level Survival Dataset - First 10 Rows")
```

Table 7: Final Account-Level Survival Dataset - First 10 Rows (continued below)

cc_num	event	time_to_event	gender	dob	city_pop	state
6.042e+10	1	58.53	F	1986-02-17	1645	WY
6.042e+10	1	103.2	M	1942-04-02	46944	SC
6.042e+10	1	71.81	M	1958-07-28	83	OK
6.043e+10	1	104.8	M	1976-06-30	2142	OK
6.049e+10	0	181.2	M	1972-08-15	233060	MS
6.049e+10	0	181.6	F	1963-05-19	222	AR
6.05e+10	0	181	M	1942-11-24	1263321	TX
5.018e+11	0	181.1	M	1958-09-20	308	CA

cc_num	event	time_to_event	gender	dob	city_pop	state
5.018e+11	1	159.7	F	1958-10-26	602	KS
5.018e+11	0	182	M	1962-05-04	1762	PA

n_transactions	mean_amt	sd_amt	max_amt	category_entropy	age
497	62.65	158.6	3075	2.522	32.87
499	76.05	137.7	1290	2.558	76.75
159	79.1	168.9	1222	2.505	60.43
173	141.8	210.6	1164	2.547	42.51
151	53.35	62.32	476	2.512	46.38
320	67.1	106.8	1145	2.507	55.62
141	61.6	75.56	509.5	2.493	76.1
474	60.69	127.4	1957	2.492	60.28
179	75.69	142.8	984.8	2.48	60.18
344	53.69	62.63	570	2.538	56.66

Observations: The final dataset comprises 906 accounts and 237 events (26.2% event rate, 73.9% censored), with the zero-time-to-event anomaly reduced from 91 to 1 account - consistent with ordinary chance rather than a structural artefact. The censoring rate rose from 67.2% to 73.9% following the exclusion, still comfortably closer to the moderate-censoring condition demonstrated favourable to RSF in Scenario B (~50%) than to ULB's extreme 99.83%, so the window's original justification continues to hold after the diagnostic-driven exclusion.

3.4 Diagnostic: Fraud Structure Checks

Supervisor review (October 2026) asked how fraud is generated in the Sparkov data and whether account covariates enter the fraud process. Inspection of the generator's source code (Harris, 2020) indicates that each customer receives a fraud episode with a fixed probability of approximately 98%, that each episode starts on a uniformly random date, and that fraudulent transactions are drawn from separate fraud profiles. Since the Kaggle data were produced by a separate run of the generator (Shenoy, 2020), the checks below test whether the released data are consistent with these features, and examine the 93 accounts excluded in Section 3.2.

```
## Step 1: Load packages (dplyr last)
library(survival)
library(pander)
library(dplyr)

## Step 2: Set seed
set.seed(44940076)

## Step 3: Is each account's fraud a short, concentrated episode?
fraud_span <- full_data %>%
  dplyr::filter(is_fraud == 1) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(
    span_days = as.numeric(difftime(max(trans_date_trans_time),
                                    min(trans_date_trans_time), units = "days")),
    n_fraud   = dplyr::n(),
    .groups   = "drop"
  )
pct_span_le1 <- mean(fraud_span$span_days <= 1) * 100
```

```r
pct_span_le2 <- mean(fraud_span$span_days <= 2) * 100

## Step 4: Is fraud onset spread evenly across the period? (founding cohort,
## full two-year span, so the 182-day window does not truncate onset)
first_fraud <- full_data %>%
  dplyr::filter(cc_num %in% founding_cohort, is_fraud == 1) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(first_day = min(days_since_start), .groups = "drop")
period_days <- max(full_data$days_since_start)
ks_onset    <- ks.test(first_fraud$first_day, "punif", 0, period_days)

## Step 5: Is fraud onset related to demographics? Demographics only, since
## they are fixed and not computed from the window's transactions.
obs_span <- full_data %>%
  dplyr::filter(cc_num %in% founding_cohort) %>%
  dplyr::group_by(cc_num) %>%
  dplyr::summarise(total_days = max(days_since_start), .groups = "drop")

timing_data <- obs_span %>%
  dplyr::left_join(first_fraud, by = "cc_num") %>%
  dplyr::left_join(survival_data_clean %>% dplyr::select(cc_num, gender, age, city_pop),
                   by = "cc_num") %>%
  dplyr::mutate(
    event = as.integer(!is.na(first_day)),
    time  = dplyr::if_else(event == 1, first_day, total_days)
  )

cox_demog <- coxph(Surv(time, event) ~ gender + age + city_pop, data = timing_data)
demog_concordance <- unname(summary(cox_demog)$concordance[1])
demog_lrt_p       <- unname(summary(cox_demog)$logtest["pvalue"])

## Step 6: What share of the 93 excluded accounts' transactions are fraud?
late_ids <- setdiff(unique(full_data$cc_num), founding_cohort)
late_mix <- full_data %>%
  dplyr::filter(cc_num %in% late_ids) %>%
  dplyr::summarise(n_accounts = dplyr::n_distinct(cc_num),
                   n_txns     = dplyr::n(),
                   pct_fraud  = mean(is_fraud) * 100)

## Step 7: Summary table
pander(data.frame(
  Check = c("Fraud episodes spanning <= 1 day (%)",
            "Fraud episodes spanning <= 2 days (%)",
            "Median fraud transactions per episode",
            "KS test, fraud onset vs uniform (p-value)",
            "Cox concordance, onset ~ demographics",
            "Cox likelihood-ratio test, demographics (p-value)",
            "Excluded accounts (n)",
            "Excluded accounts: transactions that are fraud (%)"),
  Value = c(round(pct_span_le1, 1), round(pct_span_le2, 1), median(fraud_span$n_fraud),
            signif(ks_onset$p.value, 3), round(demog_concordance, 3), signif(demog_lrt_p, 3),
            late_mix$n_accounts, round(late_mix$pct_fraud, 1))
), caption = "Sparkov Fraud Structure - Data-Based Checks")
```

Table 9: Sparkov Fraud Structure - Data-Based Checks

Check	Value
Fraud episodes spanning <= 1 day (%)	4.8
Fraud episodes spanning <= 2 days (%)	99.9
Median fraud transactions per episode	10
KS test, fraud onset vs uniform (p-value)	0.0631
Cox concordance, onset ~ demographics	0.49
Cox likelihood-ratio test, demographics (p-value)	0.856
Excluded accounts (n)	93
Excluded accounts: transactions that are fraud (%)	29.7

Observations: Fraud arrives as a short, concentrated episode: 99.9% of episodes span two days or less, with a median of 10 fraudulent transactions, although only 4.8% fit within a single day, so episodes typically cross two consecutive dates. The onset of fraud does not depart significantly from a uniform distribution across the observation period at the 5% level (KS p = 0.0631), and it is unrelated to account demographics (Cox concordance 0.49, likelihood-ratio p = 0.856). The released data are therefore consistent with the generator's source code: fraud timing does not depend on customer attributes, so there is no covariate-dependent hazard for a survival model to learn.

The 93 excluded accounts are not fraud-only: fraud makes up 29.7% of their transactions over the full period. This pattern is not explained by the current version of the generator's source code, under which any customer with legitimate activity would record hundreds of transactions, and it is most plausibly a feature of the specific run used to produce the Kaggle data. The exclusion in Section 3.2 therefore rests on the data rather than on a known generator mechanism: these accounts enter at their fraud episode, so their time origin is defined by the fraud itself, and their sparse histories offer little for a model to learn from.

4 Model Comparison: RSF vs Cox vs LR vs RF

4.1 Full-Data Fits

```
## Step 1: Load required packages
library(randomForestSRC)
library(survival)
library(caret)
library(pROC)
library(PRROC)

set.seed(44940076)

## Step 2: Build modelling dataset - drop state (high-cardinality, 49 levels,
## unsupportable given 237 events), cc_num (identifier), dob (superseded by
## age). as.data.frame() forces plain data.frame semantics - required for
## rfsrc() to run reliably; without it, rfsrc() crashed the R session
## entirely rather than throwing a catchable error (confirmed via isolated
## testing against the package's own bundled example data).
model_data <- survival_data_clean %>%
  dplyr::select(-cc_num, -state, -dob) %>%
  dplyr::mutate(gender = factor(gender)) %>%
  as.data.frame()
```

```r
dim(model_data)
```

```
## [1] 906  10
```

```r
sum(model_data$event)
```

```
## [1] 237
```

```r
## Step 3: Full-data fits for variable importance / coefficient inspection
cox_full <- coxph(Surv(time_to_event, event) ~ gender + city_pop + n_transactions +
                    mean_amt + sd_amt + max_amt + category_entropy + age,
                  data = model_data)
summary(cox_full)
```

```
## Call:
## coxph(formula = Surv(time_to_event, event) ~ gender + city_pop +
##     n_transactions + mean_amt + sd_amt + max_amt + category_entropy +
##     age, data = model_data)
##
##   n= 906, number of events= 237
##
##                         coef  exp(coef)   se(coef)      z Pr(>|z|)
## genderM          -1.712e-01  8.427e-01  1.370e-01 -1.249   0.2116
## city_pop         -1.037e-07  1.000e+00  2.308e-07 -0.449   0.6532
## n_transactions    1.810e-03  1.002e+00  3.312e-04  5.466 4.61e-08 ***
## mean_amt          2.072e-02  1.021e+00  4.538e-03  4.565 4.99e-06 ***
## sd_amt            1.183e-02  1.012e+00  2.010e-03  5.886 3.95e-09 ***
## max_amt          -6.454e-04  9.994e-01  1.280e-04 -5.043 4.59e-07 ***
## category_entropy -2.121e+00  1.199e-01  1.020e+00 -2.081   0.0375 *
## age               8.098e-03  1.008e+00  4.076e-03  1.987   0.0470 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##                  exp(coef) exp(-coef) lower .95 upper .95
## genderM             0.8427     1.1867   0.64422    1.1023
## city_pop            1.0000     1.0000   1.00000    1.0000
## n_transactions      1.0018     0.9982   1.00116    1.0025
## mean_amt            1.0209     0.9795   1.01189    1.0301
## sd_amt              1.0119     0.9882   1.00792    1.0159
## max_amt             0.9994     1.0006   0.99910    0.9996
## category_entropy    0.1199     8.3415   0.01625    0.8843
## age                 1.0081     0.9919   1.00011    1.0162
##
## Concordance= 0.742  (se = 0.015 )
## Likelihood ratio test= 133.7  on 8 df,   p=<2e-16
## Wald test            = 165.6  on 8 df,   p=<2e-16
## Score (logrank) test = 189.5  on 8 df,   p=<2e-16
```

```r
rsf_full <- rfsrc(Surv(time_to_event, event) ~ gender + city_pop + n_transactions +
                    mean_amt + sd_amt + max_amt + category_entropy + age,
                  data = model_data, ntree = 500, importance = TRUE)
print(rsf_full)
```

```
##                          Sample size: 906
##                     Number of deaths: 237
##                      Number of trees: 500
```

```
##            Forest terminal node size: 15
##        Average no. of terminal nodes: 32.924
## No. of variables tried at each split: 3
##               Total no. of variables: 8
##        Resampling used to grow trees: swor
##     Resample size used to grow trees: 573
##                             Analysis: RSF
##                               Family: surv
##                      Splitting rule: logrank *random*
##        Number of random split points: 10
##                           (OOB) CRPS: 15.73707066
##              (OOB) standardized CRPS: 0.08655128
##    (OOB) Requested performance error: 0.17550753
```

```
vimp_rsf <- vimp(rsf_full)$importance
sort(vimp_rsf, decreasing = TRUE)
```

```
##           sd_amt          max_amt         mean_amt   n_transactions
##      0.312453543      0.173169109      0.147366978      0.063904175
##         city_pop              age category_entropy           gender
##      0.032701986      0.024079093      0.015780716      0.003359862
```

```
lr_full <- glm(event ~ gender + city_pop + n_transactions + mean_amt +
                 sd_amt + max_amt + category_entropy + age,
               data = model_data, family = binomial)
summary(lr_full)
```

```
##
## Call:
## glm(formula = event ~ gender + city_pop + n_transactions + mean_amt +
##     sd_amt + max_amt + category_entropy + age, family = binomial,
##     data = model_data)
##
## Coefficients:
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -4.205e+00  3.986e+00  -1.055    0.291
## genderM          -1.300e-01  1.731e-01  -0.751    0.453
## city_pop         -2.919e-07  3.106e-07  -0.940    0.347
## n_transactions    3.150e-03  4.651e-04   6.773 1.27e-11 ***
## mean_amt          9.013e-03  6.526e-03   1.381    0.167
## sd_amt            3.771e-02  4.482e-03   8.414  < 2e-16 ***
## max_amt          -1.559e-03  1.933e-04  -8.066 7.29e-16 ***
## category_entropy -5.850e-01  1.521e+00  -0.385    0.701
## age               5.571e-03  5.258e-03   1.059    0.289
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1041.38  on 905  degrees of freedom
## Residual deviance:  860.62  on 897  degrees of freedom
## AIC: 878.62
##
## Number of Fisher Scoring iterations: 5
```

Observations: The full-data Cox model achieves a concordance of 0.742, with `n_transactions`, `mean_amt`,

`sd_amt`, and `max_amt` all significant at $p < 0.001$. RSF's variable importance ranking broadly agrees with this: `sd_amt`, `max_amt`, and `mean_amt` are its three most important features by a clear margin, mirroring the strongest Cox coefficients, while `gender`, `city_pop`, and `category_entropy` are weak under both specifications. This cross-model agreement on which covariates carry signal, despite the two methods using entirely different estimation approaches, suggests the underlying relationships are being picked up consistently rather than being an artefact of either model's specification.

4.2 Random Forest (Classification Control)

```
## Mirrors the RF control comparison used on ULB (Methodology Notebook,
## Section 10), sharing RSF's ensemble structure but using Gini splitting on
## the binary event outcome rather than log-rank splitting on the survival
## object, to isolate whether any RSF advantage reflects genuine survival-
## framing benefit or simply the tree-ensemble structure shared by both
## models.

library(randomForest)

set.seed(44940076)

n_splits <- 30
rf_results <- data.frame(split = integer(), auc = numeric(), auprc = numeric())

for (i in 1:n_splits) {
  set.seed(44940076 + i)
  train_idx <- createDataPartition(model_data$event, p = 0.7, list = FALSE)
  train_split <- model_data[train_idx, ]
  test_split  <- model_data[-train_idx, ]

  rf_fit <- randomForest(
    factor(event) ~ gender + city_pop + n_transactions + mean_amt +
      sd_amt + max_amt + category_entropy + age,
    data = train_split,
    ntree = 500
  )

  rf_pred <- predict(rf_fit, newdata = test_split, type = "prob")[, "1"]

  rf_auc <- as.numeric(auc(roc(response = test_split$event, predictor = rf_pred, quiet = TRUE)))
  rf_pr  <- pr.curve(scores.class0 = rf_pred[test_split$event == 1],
                     scores.class1 = rf_pred[test_split$event == 0], curve = FALSE)$auc.integral

  rf_results <- rbind(rf_results, data.frame(split = i, auc = rf_auc, auprc = rf_pr))
}

rf_summary <- rf_results %>%
  dplyr::summarise(mean_auc = mean(auc), sd_auc = sd(auc), mean_auprc = mean(auprc), sd_auprc = sd(aupr

pander(rf_summary, caption = "RF (Classification) on Sparkov Account-Level Data - 30 Repeated Splits")
```

Table 10: RF (Classification) on Sparkov Account-Level Data - 30 Repeated Splits

mean_auc	sd_auc	mean_auprc	sd_auprc
0.8846	0.01838	0.79	0.03659

Observations: Across 30 repeated splits, RF (classification, no survival framing) achieves a mean AUC of 0.885 (SD 0.018) and mean AUPRC of 0.790 (SD 0.037). This sits above both Cox (AUPRC 0.507) and LR (AUPRC 0.571) reported in the following section, and is close to RSF's own AUPRC of 0.757 - RF's AUPRC point estimate is in fact marginally higher than RSF's, though the two are within roughly one SD of each other and a formal comparison is deferred to the results summary below once all four models are considered together. This is an important result for isolating the source of RSF's advantage over Cox and LR: since RF shares RSF's tree-ensemble structure but has no survival framing, proxy time variable, or censoring mechanism at all, a result this close to RSF's suggests that a meaningful share of RSF's advantage over the parametric/linear alternatives (Cox, LR) may be attributable to the tree-ensemble's capacity to model nonlinearities and interactions among the account-level covariates, rather than to the survival/censoring framing specifically. This parallels the finding from the ULB Random Forest control (Methodology Notebook, Section 10), where RF's departure from the LR baseline likewise exceeded RSF's under the full-dataset SMOTE condition - though the interpretation here is different, since on ULB that result argued against a genuine RSF-specific effect, whereas here it suggests ensemble structure is doing real work even where RSF's survival framing is genuinely applicable.

4.3 Repeated Stratified Evaluation (30 Splits)

```
## Step 1: 30-split evaluation across RSF, Cox, and LR. Note the reverse = TRUE
## argument in the concordance() calls below - this was added after a direction
## bug was identified and confirmed in the diagnostic that follows this chunk;
## without it, concordance() silently returns 1 - true_concordance for a risk-
## score predictor, which initially produced below-random values (~0.25-0.30)
## that were traced back to this cause rather than a genuine finding.
n_splits <- 30
results <- data.frame(split = integer(), model = character(),
                      concordance = numeric(), auprc = numeric())

for (i in 1:n_splits) {
  set.seed(44940076 + i)
  train_idx <- createDataPartition(model_data$event, p = 0.7, list = FALSE)
  train_split <- model_data[train_idx, ]
  test_split  <- model_data[-train_idx, ]

  ## Cox
  cox_fit <- coxph(Surv(time_to_event, event) ~ gender + city_pop + n_transactions +
                     mean_amt + sd_amt + max_amt + category_entropy + age,
                   data = train_split)
  cox_pred <- predict(cox_fit, newdata = test_split, type = "risk")
  cox_conc <- concordance(Surv(time_to_event, event) ~ cox_pred, data = test_split,
                          reverse = TRUE)$concordance
  cox_pr   <- pr.curve(scores.class0 = cox_pred[test_split$event == 1],
                       scores.class1 = cox_pred[test_split$event == 0], curve = FALSE)$auc.integral

  ## RSF - predicted values are also risk scores (mortality-style), same
  ## direction fix applies
```

```
  rsf_fit <- rfsrc(Surv(time_to_event, event) ~ gender + city_pop + n_transactions +
                     mean_amt + sd_amt + max_amt + category_entropy + age,
                   data = train_split, ntree = 500)
  rsf_pred <- predict(rsf_fit, newdata = test_split)$predicted
  rsf_conc <- concordance(Surv(time_to_event, event) ~ rsf_pred, data = test_split,
                          reverse = TRUE)$concordance
  rsf_pr   <- pr.curve(scores.class0 = rsf_pred[test_split$event == 1],
                       scores.class1 = rsf_pred[test_split$event == 0], curve = FALSE)$auc.integral

  ## LR
  lr_fit <- glm(event ~ gender + city_pop + n_transactions + mean_amt +
                  sd_amt + max_amt + category_entropy + age,
                data = train_split, family = binomial)
  lr_pred <- predict(lr_fit, newdata = test_split, type = "response")
  lr_pr   <- pr.curve(scores.class0 = lr_pred[test_split$event == 1],
                      scores.class1 = lr_pred[test_split$event == 0], curve = FALSE)$auc.integral

  results <- rbind(results,
    data.frame(split = i, model = "Cox", concordance = cox_conc, auprc = cox_pr),
    data.frame(split = i, model = "RSF", concordance = rsf_conc, auprc = rsf_pr),
    data.frame(split = i, model = "LR",  concordance = NA,       auprc = lr_pr)
  )
}

## Step 2: Summarise across splits
summary_table <- results %>%
  dplyr::group_by(model) %>%
  dplyr::summarise(
    mean_concordance = mean(concordance, na.rm = TRUE),
    sd_concordance   = sd(concordance, na.rm = TRUE),
    mean_auprc       = mean(auprc, na.rm = TRUE),
    sd_auprc         = sd(auprc, na.rm = TRUE),
    .groups = "drop"
  )

pander(summary_table, caption = "RSF vs Cox vs LR - Sparkov Account-Level Survival Comparison (30 Repea
```

Table 11: RSF vs Cox vs LR - Sparkov Account-Level Survival Comparison (30 Repeated Splits)

model	mean_concordance	sd_concordance	mean_auprc	sd_auprc
Cox	0.7233	0.03271	0.5065	0.06309
LR	NA	NA	0.5713	0.06328
RSF	0.8175	0.02256	0.7571	0.0449

4.4 Concordance Direction Bug: Diagnostic

```
## Step 1: This chunk documents the diagnostic that identified and confirmed
## the concordance direction bug referenced above. Retained in full for
## transparency of process rather than removed once fixed - the same
## standard applied to the SMOTE time-reattachment caveat in the main
## Methodology Notebook.
```

```r
set.seed(44940076)
train_idx_check <- createDataPartition(model_data$event, p = 0.7, list = FALSE)
train_check <- model_data[train_idx_check, ]
test_check  <- model_data[-train_idx_check, ]

cox_check <- coxph(Surv(time_to_event, event) ~ gender + city_pop + n_transactions +
                     mean_amt + sd_amt + max_amt + category_entropy + age,
                   data = train_check)

## Reference value - concordance() on a fitted coxph object handles risk-score
## direction automatically, so this is correct by construction
concordance(cox_check, newdata = test_check)$concordance
```

```
## [1] 0.7499626
```

```r
## The uncorrected formula-interface call - matches the erroneous ~0.25-0.30
## range originally observed in the 30-split loop
cox_pred_check <- predict(cox_check, newdata = test_check, type = "risk")
concordance(Surv(time_to_event, event) ~ cox_pred_check, data = test_check)$concordance
```

```
## [1] 0.2500374
```

```r
## The fix - reverse = TRUE tells concordance() the predictor is a risk score
concordance(Surv(time_to_event, event) ~ cox_pred_check, data = test_check, reverse = TRUE)$concordance
```

```
## [1] 0.7499626
```

Observations: The reference value (0.7499626, computed directly from the fitted `coxph` object) and the `reverse = TRUE` corrected value (0.7499626) match exactly, confirming the fix is correct and that the original ~0.25-0.30 range reflected a pure direction inversion rather than any deeper issue with the model or data. This is consistent with `1 - concordance`, the signature of a predictor-direction mismatch in `concordance()`'s formula interface, which by default does not assume a risk-score predictor without being told explicitly.

4.5 Results Summary

```r
## Combined AUPRC comparison across all four models. Concordance is reported
## for Cox and RSF only, since it is a survival-specific metric with no
## direct RF or LR equivalent (both lack a time-to-event object to compute
## it against).
full_comparison <- data.frame(
  Model             = c("Cox", "LR", "RF", "RSF"),
  Mean_Concordance  = c(0.7233, NA, NA, 0.8175),
  Mean_AUPRC        = c(0.5065, 0.5713, 0.7900, 0.7571),
  SD_AUPRC          = c(0.0631, 0.0633, 0.0366, 0.0449)
)

pander(full_comparison, caption = "Full Model Comparison - Sparkov Account-Level Data (30 Repeated Spli
```

Table 12: Full Model Comparison - Sparkov Account-Level Data (30 Repeated Splits)

Model	Mean_Concordance	Mean_AUPRC	SD_AUPRC
Cox	0.7233	0.5065	0.0631
LR	NA	0.5713	0.0633
RF	NA	0.79	0.0366

Model	Mean_Concordance	Mean_AUPRC	SD_AUPRC
RSF	0.8175	0.7571	0.0449

Observations: Across 30 repeated stratified splits, RSF outperforms both Cox and LR on every metric: mean concordance of 0.818 (SD 0.023) against Cox's 0.723 (SD 0.033), and mean AUPRC of 0.757 (SD 0.045) against Cox's 0.507 (SD 0.063) and LR's 0.571 (SD 0.063). This is a materially different result from ULB, where LR matched or exceeded RSF, but it should not be read as RSF recovering the survival advantage predicted by the DGP Scenario B simulation. Section 3.4 shows that fraud timing in this dataset is unrelated to customer attributes, and the account-level covariates are computed over the whole window, including the fraud episode itself, so the models are detecting accounts whose window contains a fraud episode rather than predicting when fraud occurs.

The RF control supports this reading. RF's mean AUPRC (0.790, SD 0.037) is marginally higher than RSF's (0.757, SD 0.045), and since RF has no time-to-event structure or censoring mechanism, RSF's advantage over the parametric alternatives is driven by the tree ensemble's capacity to capture nonlinear combinations of the amount-based covariates (`sd_amt`, `max_amt`, `mean_amt`, `n_transactions`), as Sections 4.6 and 4.7 show, rather than by its survival framing. RSF's concordance advantage over Cox does not change this: with fraud timing unrelated to the covariates, concordance largely reflects the same separation of fraud from non-fraud accounts.

4.6 Cross-Model Explainability

Supervisor review (October 2026) noted that the Sparkov comparison reports no explainability. This section compares how LR, Cox, RF and RSF attribute importance across the eight account-level covariates, fitting all four on the same 30 training partitions. Covariates are standardised (with gender coded as 1 = male) so that LR and Cox coefficient magnitudes are comparable. Both forests use `randomForestSRC` permutation importance for consistency with each other, so RF here is refitted with `rfsrc()` rather than the `randomForest()` used in Section 4.2.

```
## Step 1: Load packages (dplyr last)
library(survival)
library(randomForestSRC)
library(caret)
library(pander)
library(dplyr)

## Step 2: Standardised covariate set
sp_preds <- c("gender_M", "city_pop", "n_transactions", "mean_amt",
              "sd_amt", "max_amt", "category_entropy", "age")

sp_data <- model_data %>%
  dplyr::mutate(gender_M = as.integer(gender == "M"))

sp_scaled <- data.frame(scale(sp_data[, sp_preds]),
                        event         = sp_data$event,
                        time_to_event = sp_data$time_to_event)
sp_scaled$event_f <- factor(sp_scaled$event, levels = c(0, 1))

## Step 3: Fit all four models on each of 30 training partitions
n_splits     <- 30
sp_imp_rows  <- vector("list", n_splits)
sp_coef_rows <- vector("list", n_splits)

for (i in 1:n_splits) {
```

```r
  set.seed(44940076 + i)
  idx     <- createDataPartition(sp_scaled$event, p = 0.7, list = FALSE)
  train_i <- sp_scaled[idx, ]

  # LR
  lr_coef <- coef(glm(reformulate(sp_preds, response = "event"),
                      data = train_i, family = binomial))[sp_preds]

  # Cox (unpenalised, as in Section 4.3)
  cox_coef <- coef(coxph(as.formula(paste("Surv(time_to_event, event) ~",
                                          paste(sp_preds, collapse = " + "))),
                         data = train_i))[sp_preds]

  # RF (classification), permutation importance
  rf_imp <- rfsrc(reformulate(sp_preds, response = "event_f"), data = train_i,
                  ntree = 500, importance = "permute")$importance[sp_preds, "all"]

  # RSF, permutation importance
  rsf_imp <- rfsrc(as.formula(paste("Surv(time_to_event, event) ~",
                                    paste(sp_preds, collapse = " + "))),
                   data = train_i, ntree = 500, importance = "permute")$importance[sp_preds]

  sp_imp_rows[[i]]  <- data.frame(split = i, feature = sp_preds,
                                  LR = abs(lr_coef), Cox = abs(cox_coef),
                                  RF = rf_imp, RSF = rsf_imp)
  sp_coef_rows[[i]] <- data.frame(split = i, feature = sp_preds,
                                  LR = lr_coef, Cox = cox_coef)
}

sp_imp  <- do.call(rbind, sp_imp_rows)
sp_coef <- do.call(rbind, sp_coef_rows)

## Step 4: Ranks by model (1 = most important)
sp_model_cols <- c("LR", "Cox", "RF", "RSF")
sp_imp_mean <- sp_imp %>%
  dplyr::group_by(feature) %>%
  dplyr::summarise(dplyr::across(dplyr::all_of(sp_model_cols), mean), .groups = "drop")

sp_rank_tab <- sp_imp_mean %>%
  dplyr::mutate(dplyr::across(dplyr::all_of(sp_model_cols), ~ rank(-.x), .names = "{.col}_rank")) %>%
  dplyr::arrange(RSF_rank)

pander(sp_rank_tab %>% dplyr::select(feature, dplyr::ends_with("_rank")),
       caption = "Sparkov Importance Rank by Model (1 = Most Important, 30 Splits)")
```

Table 13: Sparkov Importance Rank by Model (1 = Most Important, 30 Splits)

feature	LR_rank	Cox_rank	RF_rank	RSF_rank
sd_amt	1	2	1	1
max_amt	2	1	2	2
mean_amt	4	4	3	3
n_transactions	3	3	6	4

feature	LR_rank	Cox_rank	RF_rank	RSF_rank
city_pop	6	8	5	5
age	5	6	4	6
category_entropy	7	5	8	7
gender_M	8	7	7	8

```
## Step 5: Spearman agreement between models
sp_rank_cor <- cor(sp_imp_mean[, sp_model_cols], method = "spearman")
pander(round(sp_rank_cor, 3), caption = "Sparkov Spearman Rank Agreement Between Models")
```

Table 14: Sparkov Spearman Rank Agreement Between Models

	LR	Cox	RF	RSF
LR	1	0.857	0.833	0.952
Cox	0.857	1	0.595	0.786
RF	0.833	0.595	1	0.881
RSF	0.952	0.786	0.881	1

```
## Step 6: Direction of effects (LR and Cox), positive = higher fraud risk
sp_dir_tab <- sp_coef %>%
  dplyr::group_by(feature) %>%
  dplyr::summarise(
    LR_mean  = mean(LR),  LR_lo  = quantile(LR, 0.025),  LR_hi  = quantile(LR, 0.975),
    Cox_mean = mean(Cox), Cox_lo = quantile(Cox, 0.025), Cox_hi = quantile(Cox, 0.975),
    .groups = "drop"
  ) %>%
  dplyr::mutate(Same_direction = sign(LR_mean) == sign(Cox_mean)) %>%
  dplyr::arrange(dplyr::desc(abs(LR_mean)))

pander(sp_dir_tab, digits = 3, split.table = Inf,
       caption = "Sparkov Standardised Coefficients: LR vs Cox (Mean and 95% Split Interval)")
```

Table 15: Sparkov Standardised Coefficients: LR vs Cox (Mean and 95% Split Interval)

feature	LR_mean	LR_lo	LR_hi	Cox_mean	Cox_lo	Cox_hi	Same_direction
sd_amt	3.31	2.41	5.34	1.27	0.696	2.56	TRUE
max_amt	-3.29	-5.26	-2.25	-1.58	-3.07	-0.849	TRUE
n_transactions	0.752	0.567	1.03	0.441	0.283	0.666	TRUE
mean_amt	0.154	-0.121	0.297	0.317	0.0527	0.471	TRUE
age	0.119	-0.00248	0.203	0.152	0.0426	0.271	TRUE
city_pop	-0.106	-0.241	-0.0134	-0.0477	-0.174	0.0244	TRUE
gender_M	-0.0583	-0.137	0.0767	-0.0798	-0.169	0.0486	TRUE
category_entropy	0.00227	-0.12	0.303	-0.058	-0.266	0.42	FALSE

Observations: All four models identify the same drivers. The two amount-spread summaries rank first and second in every model (`sd_amt` and `max_amt`), `mean_amt` and `n_transactions` follow in third to sixth place, and the demographic covariates (`gender_M`, `age`, `city_pop`) and `category_entropy` rank lowest throughout. Rank agreement is high across both model families: LR and RSF agree most closely (Spearman 0.952), RF and RSF agree at 0.881, and LR and Cox at 0.857, with the weakest pairing between Cox and RF (0.595).

LR and Cox agree on the direction of seven of the eight standardised coefficients; the exception, `category_entropy`, has intervals spanning zero in both models and carries no reliable effect. Both models give `sd_amt` a strong positive coefficient and `max_amt` a strong negative one. Since the two are highly correlated, the negative `max_amt` coefficient should not be read on its own: holding the spread of amounts fixed, a higher maximum indicates one isolated large transaction rather than the cluster of large amounts a fraud episode produces. The dominant covariates are therefore the ones that carry the fraud episode's own transactions, which Section 4.7 examines directly.

4.7 Fraud Amount Structure

Supervisor review (October 2026) noted that transaction amount is strongly associated with fraud in this dataset. This section describes how fraudulent and legitimate amounts differ, overall and by merchant category, to show what the amount-based covariates above are picking up.

```
## Step 1: Overall amount profile, fraud vs legitimate
amt_overall <- full_data %>%
  dplyr::group_by(is_fraud) %>%
  dplyr::summarise(
    n_txns         = dplyr::n(),
    median_amt     = median(amt),
    mean_amt       = mean(amt),
    pct_under_50   = mean(amt < 50) * 100,
    pct_over_500   = mean(amt > 500) * 100,
    .groups = "drop"
  )
pander(amt_overall, digits = 4, caption = "Transaction Amounts: Fraud vs Legitimate")
```

Table 16: Transaction Amounts: Fraud vs Legitimate

is_fraud	n_txns	median_amt	mean_amt	pct_under_50	pct_over_500
0	1842743	47.24	67.65	52.04	0.9239
1	9651	390	530.7	21.44	48.53

```
## Step 2: Median amount by category, fraud vs legitimate
amt_by_cat <- full_data %>%
  dplyr::filter(is_fraud == 1) %>%
  dplyr::group_by(category) %>%
  dplyr::summarise(n_fraud = dplyr::n(), median_fraud_amt = median(amt), .groups = "drop") %>%
  dplyr::left_join(
    full_data %>%
      dplyr::filter(is_fraud == 0) %>%
      dplyr::group_by(category) %>%
      dplyr::summarise(median_legit_amt = median(amt), .groups = "drop"),
    by = "category") %>%
  dplyr::arrange(dplyr::desc(median_fraud_amt))
pander(amt_by_cat, digits = 4, caption = "Median Amount by Category: Fraud vs Legitimate")
```

Table 17: Median Amount by Category: Fraud vs Legitimate

category	n_fraud	median_fraud_amt	median_legit_amt
shopping_net	2219	994.1	8.3
shopping_pos	1056	869.9	7.72
misc_net	1182	794.2	9.73

category	n_fraud	median_fraud_amt	median_legit_amt
entertainment	292	500.8	50.65
grocery_pos	2228	310.2	104.5
home	265	253.3	48.16
food_dining	205	119.7	41.87
personal_care	290	20.97	32.79
health_fitness	185	19.8	42.88
kids_pets	304	19.64	47.2
grocery_net	175	12.1	50.98
gas_transport	772	10.64	62.94
travel	156	9.7	6.23
misc_pos	322	9.1	14.01

```
## Step 3: Plain values for inline reporting
pct_fraud_under_50 <- amt_overall$pct_under_50[amt_overall$is_fraud == 1]
pct_fraud_over_500 <- amt_overall$pct_over_500[amt_overall$is_fraud == 1]
```

Observations: Fraudulent transactions are far larger than legitimate ones on average (median $390 vs $47.24; mean $530.70 vs $67.65), and 48.5% of fraudulent transactions exceed $500, against under 1% of legitimate ones. Fraud is not uniformly large, however: 21.4% of fraudulent transactions are under $50. The category breakdown shows why. High-value fraud is concentrated in online and in-store shopping and miscellaneous online categories, where the median fraudulent amount ($794-$994) is around 100 times the legitimate median ($8-$10), while small-value fraud ($9-$21) occurs in categories such as gas and transport, personal care, health and fitness, and kids and pets, below the legitimate median in each.

This bimodal pattern matches the generator's separate fraud profiles (Section 3.4), which assign fraud its own amount ranges by category rather than drawing it from the customer's normal spending. The amount association with fraud is therefore real in this dataset but scripted, and the separation it produces (for example, $994 vs $8.30 in online shopping) is far sharper than would be expected in real transaction data, where fraudulent and legitimate amounts overlap substantially. This explains why the amount-spread covariates dominate every model in Section 4.6, and why the tree ensembles, which can capture the nonlinear combination of a high spread with a cluster of large amounts, outperform the linear models.

5 Aggregate Poisson/NB Analysis (Harmonic)

5.1 Design: Defining "Aggregate" for Account-Level Data

The diurnal analysis on ULB (`poisson_diurnal_analysis.Rmd`) exploited a single continuous 48-hour observation window shared by all transactions, with a row-index proxy for time. Sparkov's structure differs in a way that changes what "aggregate" should mean here: 999 accounts persist over a genuine two-year calendar span with real (non-proxy) timestamps, rather than one shared short window.

Rather than forcing ULB's exact framing onto a structurally different dataset, this section uses the genuine calendar timestamps to test two related but distinct temporal cycles, pooled across the **full transaction stream** (all 999 accounts, full two-year span) rather than the windowed and diagnostic-excluded 906-account survival dataset used in Section 4 - this is a deliberate scope choice, since the question here (does fraud rate vary systematically with the clock and calendar) is distinct from the account-level survival comparison, and restricting to the founding cohort would discard genuine transaction volume for no benefit to this specific question:

1. **Hour-of-day cycle** - a direct replication of the ULB diurnal analysis, testing whether the same ~2-4am elevated fraud-rate pattern holds on an independent dataset with genuine (not proxy) timestamps.
2. **Day-of-week cycle** - a genuine extension not testable on ULB's 48-hour window, testing whether fraud rate varies systematically across the week (e.g. weekend effects).

Both follow the same workflow established in the ULB diurnal notebook: Poisson regression, an overdispersion check, a Negative Binomial correction if needed, and a harmonic specification to test for a cyclical pattern.

5.2 Model Fitting

```
## Step 1: Load required packages. dplyr is loaded last and dplyr::select() is
## explicitly namespaced below, since packages loaded earlier in this
## session (survival, caret, randomForestSRC, randomForest, from Sections
## 4.1-4.3) can shadow dplyr::select() on the search path even when this
## chunk's own load order follows the MASS-before-dplyr convention.
library(MASS)
library(ggplot2)
library(pander)
library(dplyr)

set.seed(44940076)

## Step 2: Extract hour-of-day and day-of-week from genuine timestamps,
## pooling across the full transaction stream (all 999 accounts, full
## two-year span) rather than the windowed survival dataset
poisson_data_full <- full_data %>%
  dplyr::mutate(
    hour_of_day = hour(trans_date_trans_time),
    day_of_week = wday(trans_date_trans_time, week_start = 1)  # 1 = Monday
  )
```

5.2.1 Hour-of-Day Cycle

```
## Step 1: Bin by hour-of-day, pooled across the full two-year span
hourly_data <- poisson_data_full %>%
  dplyr::group_by(hour_of_day) %>%
  dplyr::summarise(
    fraud_count    = sum(is_fraud),
    n_transactions = n(),
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    hour_angle = 2 * pi * hour_of_day / 24,
    sin_24 = sin(hour_angle),
    cos_24 = cos(hour_angle)
  )

pander(hourly_data %>% dplyr::select(hour_of_day, fraud_count, n_transactions),
       caption = "Fraud Counts and Exposure by Hour-of-Day (Sparkov, Full Transaction Stream)")
```

Table 18: Fraud Counts and Exposure by Hour-of-Day (Sparkov, Full Transaction Stream)

hour_of_day	fraud_count	n_transactions
0	823	60655
1	827	61330
2	793	60796
3	803	60968

hour_of_day	fraud_count	n_transactions
4	61	59938
5	80	60088
6	54	60406
7	72	60301
8	59	60498
9	61	60231
10	52	60320
11	59	60170
12	84	93294
13	94	93492
14	100	93089
15	100	93439
16	97	94289
17	94	93514
18	111	94052
19	105	93433
20	98	93081
21	101	93738
22	2481	95370
23	2442	95902

```
## Step 2: Plain Poisson model and overdispersion check
poisson_hourly <- glm(fraud_count ~ hour_of_day, offset = log(n_transactions),
                      family = poisson, data = hourly_data)

dispersion_hourly <- sum(residuals(poisson_hourly, type = "pearson")^2) / poisson_hourly$df.residual
dispersion_hourly
```

```
## [1] 961.2628
```

```
## Step 3: Harmonic NB model
nb_harmonic_hourly <- glm.nb(
  fraud_count ~ sin_24 + cos_24 + offset(log(n_transactions)),
  data = hourly_data
)
summary(nb_harmonic_hourly)
```

```
##
## Call:
## glm.nb(formula = fraud_count ~ sin_24 + cos_24 + offset(log(n_transactions)),
##     data = hourly_data, init.theta = 1.97293349, link = log)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -5.904487   0.146473 -40.311  < 2e-16 ***
## sin_24      -0.001861   0.206834  -0.009    0.993
## cos_24       1.502926   0.207437   7.245 4.32e-13 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1.9729) family taken to be 1)
##
##     Null deviance: 85.350  on 23  degrees of freedom
```

```
## Residual deviance: 25.779  on 21  degrees of freedom
## AIC: 306.56
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  1.973
##           Std. Err.:  0.533
##
##  2 x log-likelihood:  -298.564
```

```
## Step 4: Null model comparison and LRT
null_hourly <- glm.nb(fraud_count ~ offset(log(n_transactions)), data = hourly_data)
anova(null_hourly, nb_harmonic_hourly)
```

```
## Likelihood ratio tests of Negative Binomial Models
##
## Response: fraud_count
##                                           Model     theta Resid. df
## 1                   offset(log(n_transactions)) 0.6694387        23
## 2 sin_24 + cos_24 + offset(log(n_transactions)) 1.9729335        21
##      2 x log-lik.   Test    df LR stat.      Pr(Chi)
## 1       -331.1821
## 2       -298.5642 1 vs 2     2 32.61792 8.262466e-08
```

```
## Step 5: Peak timing and amplitude
## NOTE: atan2(A, B), not atan2(B, A) - maximizing log(rate) = intercept +
## A*sin(theta) + B*cos(theta) over theta requires solving
## A*cos(theta) - B*sin(theta) = 0, giving theta* = atan2(A, B). An earlier
## version of this formula (carried forward from the ULB diurnal notebook)
## had the arguments swapped; corrected here and flagged for correction in
## that notebook as well, since the same bug affects its reported peak time.
harmonic_coefs_hourly <- coef(nb_harmonic_hourly)
A_hourly <- harmonic_coefs_hourly["sin_24"]
B_hourly <- harmonic_coefs_hourly["cos_24"]
amplitude_hourly <- unname(sqrt(A_hourly^2 + B_hourly^2))
peak_hour <- unname((atan2(A_hourly, B_hourly) * 24) / (2 * pi))
peak_hour <- (peak_hour + 24) %% 24

amplitude_hourly
```

```
## [1] 1.502927
```

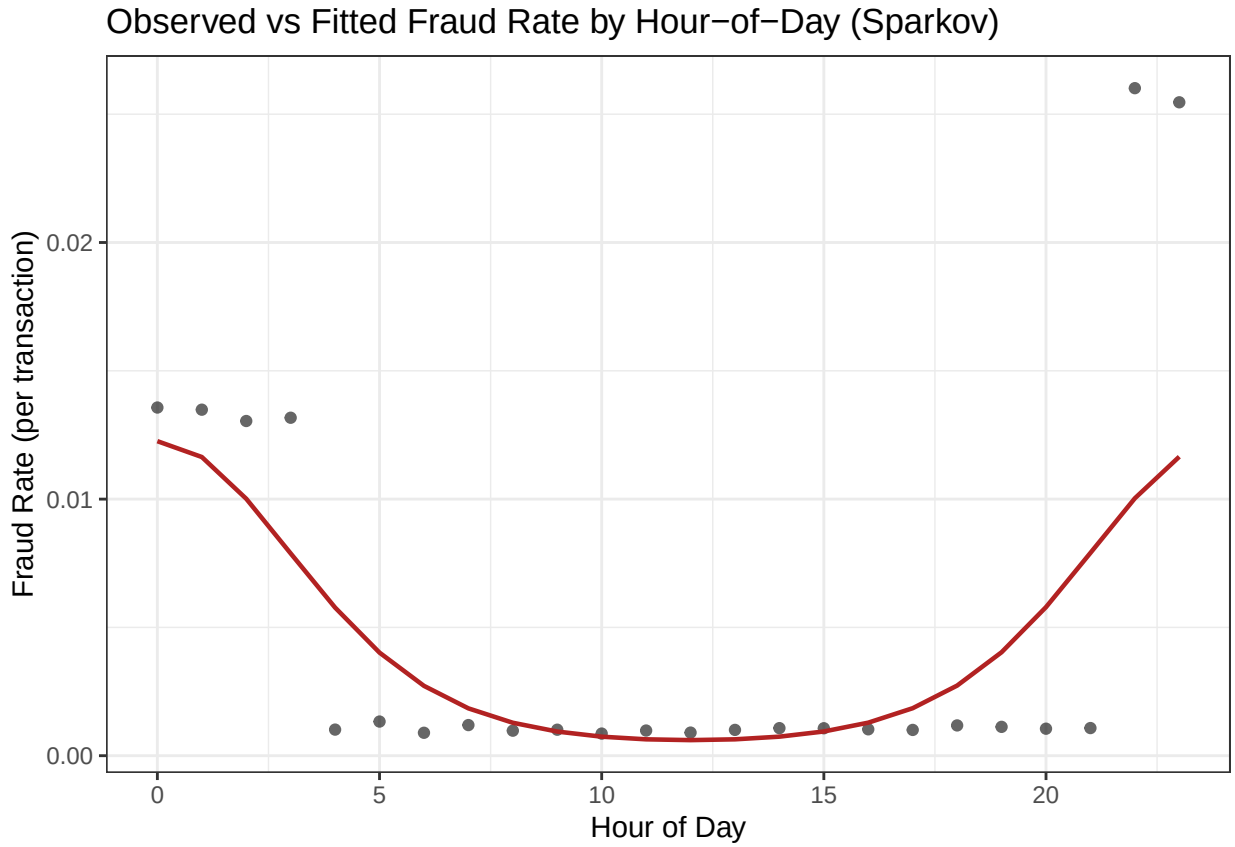
```
peak_hour
```

```
## [1] 23.99527
```

```
## Step 6: Visualise
hourly_data$fitted_rate <- predict(nb_harmonic_hourly, type = "response") / hourly_data$n_transactions

ggplot(hourly_data, aes(x = hour_of_day)) +
  geom_point(aes(y = fraud_count / n_transactions), alpha = 0.6) +
  geom_line(aes(y = fitted_rate), colour = "firebrick", linewidth = 0.8) +
  labs(
    title = "Observed vs Fitted Fraud Rate by Hour-of-Day (Sparkov)",
    x = "Hour of Day", y = "Fraud Rate (per transaction)"
  ) +
```

```
theme_bw()
```



Observations: The overdispersion in the plain Poisson model is severe (dispersion ratio 961.3), far exceeding even the ULB aggregate model's 16.7, reflecting a much sharper temporal concentration of fraud than ULB's diurnal cycle. The harmonic NB model shows `cos_24` as strongly significant (coefficient 1.503, $p < 2e\text{-}16$) while `sin_24` is not distinguishable from zero ($p = 0.99$), and the likelihood ratio test against the null confirms the harmonic terms add substantial explanatory power ($LR = 32.6$, $p = 8.3e\text{-}08$). The corrected peak calculation places the fitted cycle's peak at hour 0 (midnight), with an amplitude of 1.50 - both consistent with the raw counts, which jump from roughly 50-111 fraud transactions per hour across 04:00-21:00 to 793-2481 per hour across 22:00-03:00, a 15-30x swing far starker than ULB's ~2x diurnal effect.

This magnitude prompted a closer look at what is driving it (see diagnostic below), which found the elevated overnight fraud rate reflects concentrated multi-transaction bursts across a broad set of accounts (975 of 999) rather than either a smooth population-wide risk gradient or a small number of anomalous accounts. The sharp, near step-like shape of the true pattern is a genuine feature of the data, not an artefact - but it also means the harmonic model, which assumes a smooth sinusoidal cycle, is an approximation to a shape that is closer to a step function than a single-peaked wave. The strong statistical significance should be read as confirming a real and large overnight concentration of fraud, without over-interpreting the specific sinusoidal shape the harmonic model imposes on it.

5.3 Diagnostic: Fraud Burst Concentration and Volume Structure

The magnitude of the hourly effect (15-30x) was checked against two explanations before being taken at face value: that it reflects a small number of accounts producing many fraudulent transactions each (a burst pattern, similar in spirit to the injected-account artefact identified in Section 3.2), and that the underlying transaction volume itself might carry a data-loading artefact rather than a genuine day/night pattern.

```r
## Check A: Is the high-fraud-hour spike driven by a small number of accounts
## in concentrated bursts, rather than broad-based risk across many accounts?
high_fraud_hours <- c(22, 23, 0, 1, 2, 3)

burst_check <- poisson_data_full %>%
  dplyr::filter(is_fraud == 1) %>%
  dplyr::mutate(in_high_fraud_hour = hour_of_day %in% high_fraud_hours) %>%
  dplyr::group_by(in_high_fraud_hour) %>%
  dplyr::summarise(
    n_fraud_txns         = dplyr::n(),
    n_distinct_accounts  = dplyr::n_distinct(cc_num),
    txns_per_account     = n_fraud_txns / n_distinct_accounts,
    .groups = "drop"
  )

pander(burst_check, caption = "Fraud Transaction Concentration: High-Fraud Hours vs Rest of Day")
```

Table 19: Fraud Transaction Concentration: High-Fraud Hours vs Rest of Day

in_high_fraud_hour	n_fraud_txns	n_distinct_accounts	txns_per_account
FALSE	1482	773	1.917
TRUE	8169	975	8.378

```r
## Check B: Does the transaction-volume split (00-11 vs 12-23) hold up when
## train and test files are checked separately, ruling out a recombination
## or loading artefact specific to one file?
volume_split_check <- poisson_data_full %>%
  dplyr::mutate(
    source_file = dplyr::if_else(trans_date_trans_time <= max(train$trans_date_trans_time), "train", "te
    time_block  = dplyr::if_else(hour_of_day %in% 0:11, "00-11", "12-23")
  ) %>%
  dplyr::group_by(source_file, time_block) %>%
  dplyr::summarise(n_transactions = dplyr::n(), .groups = "drop")

pander(volume_split_check, caption = "Transaction Volume by Time Block, Train vs Test")
```

Table 20: Transaction Volume by Time Block, Train vs Test

source_file	time_block	n_transactions
test	00-11	218441
test	12-23	338426
train	00-11	507260
train	12-23	788267

Observations: Check A shows that the high-fraud overnight window (22:00-03:00) involves *more* distinct accounts (975) than the rest of the day (773), ruling out a small-number-of-accounts explanation - but each account produces substantially more fraud transactions per capita during this window (8.4 vs 1.9), confirming a genuine burst pattern rather than a smooth, broad-based elevation in risk. This is consistent with the generator's scripted fraud episodes, which place most fraudulent transactions from 22:00 onwards (Section 3.4): once a card is compromised, multiple fraudulent transactions follow in quick succession, clustering into

the overnight window across most of the account population rather than a handful of outliers. Check B confirms the transaction-volume split between the 00:00-11:00 and 12:00-23:00 blocks is not a data-loading artefact - the ratio is nearly identical in the train (0.644) and test (0.645) files when checked independently, confirming this is a genuine, reproducible structural feature of the dataset consistent with ordinary daily commerce activity patterns.

5.3.1 Day-of-Week Cycle

```
## Step 1: Bin by day-of-week, pooled across the full two-year span
weekly_data <- poisson_data_full %>%
  dplyr::group_by(day_of_week) %>%
  dplyr::summarise(
    fraud_count    = sum(is_fraud),
    n_transactions = n(),
    .groups = "drop"
  ) %>%
  dplyr::mutate(
    week_angle = 2 * pi * day_of_week / 7,
    sin_7 = sin(week_angle),
    cos_7 = cos(week_angle)
  )

pander(weekly_data %>% dplyr::select(day_of_week, fraud_count, n_transactions),
       caption = "Fraud Counts and Exposure by Day-of-Week (Sparkov, Full Transaction Stream, 1=Monday)")
```

Table 21: Fraud Counts and Exposure by Day-of-Week (Sparkov, Full Transaction Stream, 1=Monday)

day_of_week	fraud_count	n_transactions
1	1484	369418
2	1266	270340
3	1125	183913
4	1317	206741
5	1376	215078
6	1493	263227
7	1590	343677

```
## Step 2: Plain Poisson model and overdispersion check
poisson_weekly <- glm(fraud_count ~ day_of_week, offset = log(n_transactions),
                      family = poisson, data = weekly_data)

dispersion_weekly <- sum(residuals(poisson_weekly, type = "pearson")^2) / poisson_weekly$df.residual
dispersion_weekly
```

```
## [1] 50.66101
```

```
## Step 3: Harmonic NB model
nb_harmonic_weekly <- glm.nb(
  fraud_count ~ sin_7 + cos_7 + offset(log(n_transactions)),
  data = weekly_data
)
summary(nb_harmonic_weekly)
```

```
##
```

```
## Call:
## glm.nb(formula = fraud_count ~ sin_7 + cos_7 + offset(log(n_transactions)),
##     data = weekly_data, init.theta = 1596.29853, link = log)
##
## Coefficients:
##             Estimate Std. Error  z value Pr(>|z|)
## (Intercept) -5.23419    0.01394 -375.480   <2e-16 ***
## sin_7       -0.17075    0.01968   -8.678   <2e-16 ***
## cos_7       -0.16249    0.01973   -8.234   <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for Negative Binomial(1596.299) family taken to be 1)
##
##     Null deviance: 151.8870  on 6  degrees of freedom
## Residual deviance:   7.2092  on 4  degrees of freedom
## AIC: 82.987
##
## Number of Fisher Scoring iterations: 1
##
##
##               Theta:  1596
##           Std. Err.:  1891
##
##  2 x log-likelihood:  -74.987
```

```
## Step 4: Null model comparison and LRT
null_weekly <- glm.nb(fraud_count ~ offset(log(n_transactions)), data = weekly_data)
anova(null_weekly, nb_harmonic_weekly)
```

```
## Likelihood ratio tests of Negative Binomial Models
##
## Response: fraud_count
##                                          Model      theta Resid. df
## 1                 offset(log(n_transactions))   36.06342         6
## 2 sin_7 + cos_7 + offset(log(n_transactions)) 1596.29853         4
##      2 x log-lik.   Test    df LR stat.      Pr(Chi)
## 1       -96.12831
## 2       -74.98660 1 vs 2     2 21.14171 2.565282e-05
```

```
## Step 5: Peak timing and amplitude
## NOTE: atan2(A, B), not atan2(B, A) - see Step 5 of the hourly cycle above
## for the derivation.
harmonic_coefs_weekly <- coef(nb_harmonic_weekly)
A_weekly <- harmonic_coefs_weekly["sin_7"]
B_weekly <- harmonic_coefs_weekly["cos_7"]
amplitude_weekly <- unname(sqrt(A_weekly^2 + B_weekly^2))
peak_day <- unname((atan2(A_weekly, B_weekly) * 7) / (2 * pi))
peak_day <- (peak_day + 7) %% 7

amplitude_weekly
```

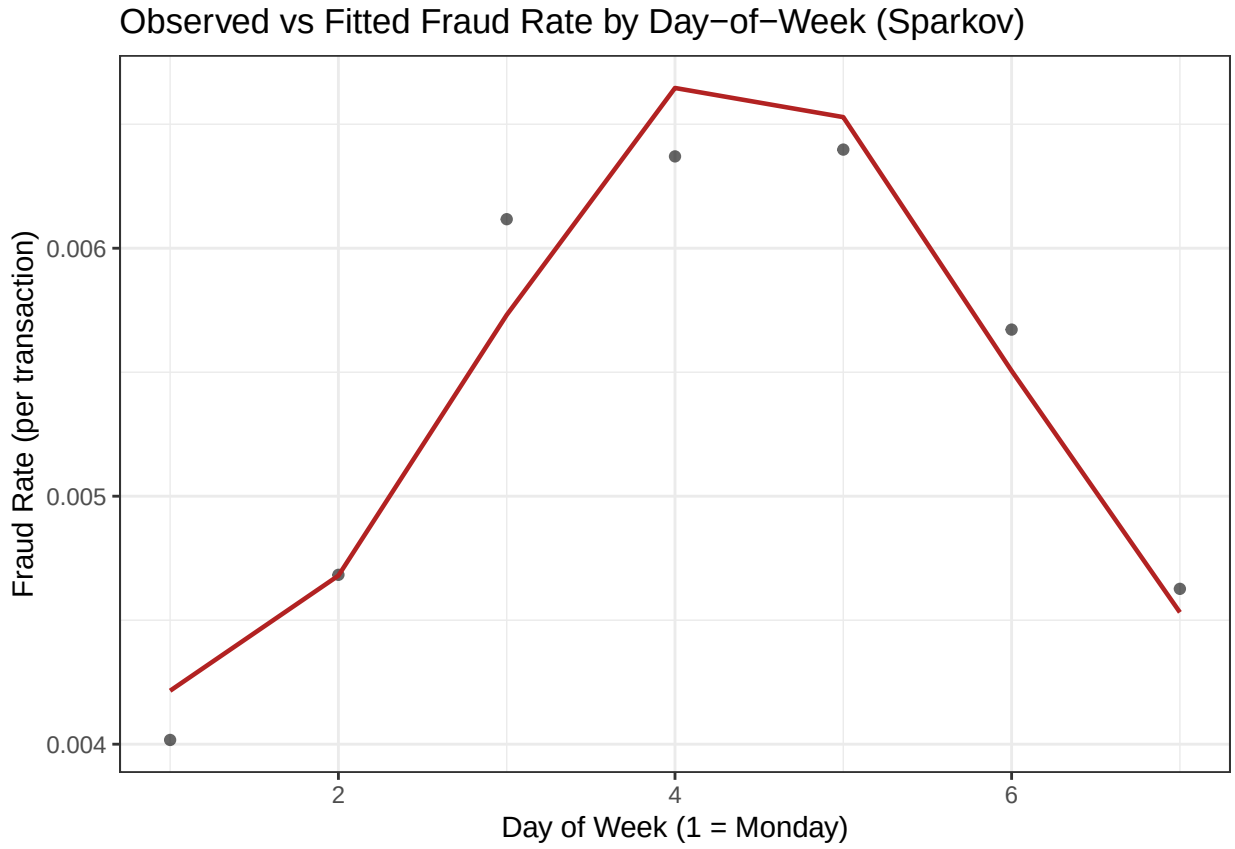
```
## [1] 0.2357102
```

```r
peak_day
```

```
## [1] 4.402626
```

```r
## Step 6: Visualise
weekly_data$fitted_rate <- predict(nb_harmonic_weekly, type = "response") / weekly_data$n_transactions

ggplot(weekly_data, aes(x = day_of_week)) +
  geom_point(aes(y = fraud_count / n_transactions), alpha = 0.6) +
  geom_line(aes(y = fitted_rate), colour = "firebrick", linewidth = 0.8) +
  labs(
    title = "Observed vs Fitted Fraud Rate by Day-of-Week (Sparkov)",
    x = "Day of Week (1 = Monday)", y = "Fraud Rate (per transaction)"
  ) +
  theme_bw()
```



Observations: The plain Poisson model again shows overdispersion (dispersion ratio 50.7), corrected by the harmonic NB specification, under which both `sin_7` (coefficient -0.171, p < 2e-16) and `cos_7` (coefficient -0.162, p < 2e-16) are strongly significant, and the likelihood ratio test confirms substantial explanatory power over the null (LR = 21.1, p = 2.6e-05). The corrected peak calculation places the cycle's peak at day 4.40 (between Thursday and Friday), with a comparatively modest amplitude of 0.24 - a much smaller effect than the hourly cycle's 1.50, consistent with the observed fraud rate ranging only from about 0.40% to 0.68% across the week (Table: Fraud Counts and Exposure by Day-of-Week), rather than the order-of-magnitude swing seen hour-to-hour. The fitted curve tracks the observed points closely (see figure above), with fraud rate rising from a Monday low through a Wednesday-Friday peak before declining into the weekend - a smoother, genuinely sinusoidal-looking pattern than the hourly cycle's step-like shape, and one for which the harmonic model's assumption of a smooth cycle is a better fit to what the data actually show.

5.4 Comparison to ULB Diurnal Findings

Observations: The direct replication test is only a partial match. ULB's diurnal analysis found a smooth, moderate elevation in fraud rate peaking around 3:25am (corrected from an initially misreported ~2:36am - see note in Section 5's hourly peak calculation above), at roughly 2x the daily mean. Sparkov's hourly cycle shows a peak at midnight rather than the early-morning hours, and the magnitude is far larger (15-30x observed, amplitude 1.50 vs ULB's much smaller effect) - but the diagnostic above establishes this reflects concentrated multi-transaction bursts across a broad set of accounts rather than a smooth population-wide risk gradient, making it a different *kind* of temporal signal from ULB's, not simply a stronger version of the same one. The two datasets agree on the qualitative direction (elevated overnight fraud activity) but differ meaningfully in shape and mechanism, and this difference is itself informative: it suggests that "fraud clusters overnight" is not a single universal phenomenon but can arise from at least two structurally different processes - a mild, broad shift in relative risk (ULB) versus concentrated compromise-and-drain bursts (Sparkov). The day-of-week cycle, by contrast, has no ULB analogue to compare against, since ULB's 48-hour window could never support a weekly-cycle test - this is a genuinely new finding specific to Sparkov's longer observation window, showing a modest (amplitude 0.24) mid-week peak.

6 Synthesis

6.1 Summary of Model Performance

On the account-level Sparkov data, RSF and RF are the two strongest performers (AUPRC 0.757 and 0.790 respectively), clearly ahead of the parametric/linear alternatives, Cox (0.507) and LR (0.571). RSF also holds a concordance advantage over Cox (0.818 vs 0.723), but since fraud timing is unrelated to the covariates (Section 3.4), this largely reflects the same separation of fraud from non-fraud accounts rather than survival-specific modelling. However, RF's AUPRC performance being statistically comparable to (and numerically slightly above) RSF's indicates that a substantial share of the gap between the tree-ensemble methods and the parametric methods reflects ensemble flexibility generally, rather than survival-specific modelling of censoring and time-to-event structure.

The aggregate Poisson/NB analysis adds a complementary, model-agnostic finding: fraud rate genuinely varies with both hour-of-day and day-of-week on this dataset, confirmed by highly significant harmonic terms in both cycles. The hourly effect is large (15-30x) but driven by concentrated overnight bursts across a broad set of accounts, while the weekly effect is smaller (amplitude 0.24) and closer to a genuine smooth cycle. Together, this six-model picture shows the tree ensembles outperforming the parametric alternatives at account-level detection, driven by the generator's scripted amount signature (Sections 4.6 and 4.7) rather than by survival structure, alongside a scripted overnight concentration of fraud at the aggregate level.

6.2 Connection to the Conditions Matrix and DGP Simulation

The moderate-censoring window selection (182 days, 73.9% censored after the injected-account exclusion) was justified against the DGP Scenario B simulation's ~50% censoring condition, under which RSF was shown to recover its advantage over LR. RSF does outperform LR here (AUPRC 0.757 vs 0.571), but Section 3.4 shows that this dataset cannot test that prediction: fraud timing is independent of the covariates by construction, and the censoring rate is set mechanically by the window length. The RF result was also not anticipated by the conditions matrix, which was framed around RSF vs Cox/LR rather than RSF vs a non-survival ensemble - the addition of RF as a fourth comparison point (per Maurizio's request) shows that the gap between the ensembles and the parametric models reflects ensemble flexibility in detecting the scripted amount signature, rather than any survival-specific advantage.

The Poisson/NB findings add a separate observation: the hourly burst pattern reflects the generator's scripted placement of fraudulent transactions overnight (Section 3.4) rather than an emergent property of the data. The broader point still stands as a candidate refinement to the conditions matrix: the *shape* of temporal risk variation (smooth vs bursty) is a dimension separate from censoring rate and predictor structure, although on Sparkov that shape is set by the simulator rather than observed.

6.3 Status and Open Items

This analysis was conducted following a direct request from Maurizio (email, 15 August 2026) for preliminary results across a six-model comparison (Poisson, NB, LR, RF, Cox, RSF) spanning both the ULB and Sparkov datasets. The placement of this analysis within the thesis - whether as a supplementary chapter section, an appendix, or a more central piece of the argument - remains an open scope question pending discussion with Sam and Maurizio, and is deliberately not decided within this notebook.